计算机科学与技术学院 神经网络与深度学习课程实验报告

实验题目：全连接网络实验		学号：202200130077
日期：2024.10.8	班级：22 级人工智能	姓名：薛润泽
Email：		

实验目的：

通过本次实验，掌握全连接网络模型的构建及训练方法，熟悉 PyTorch 深度学习框架的基本操作，包括数据集加载、模型定义、损失函数和优化器的选择以及模型的训练和评估。

实验软件和硬件环境：

Windows，Python 语言

实验原理和方法：

全连接网络（Fully Connected Neural Network, FCNN）是一种基本的神经网络结构，其中每一层的神经元都与下一层的所有神经元相连。在本实验中，使用 PyTorch 框架构建了一个具有两个隐藏层的全连接网络模型，用于 MNIST 手写数字识别任务。MNIST 数据集包含 0 到 9 的手写数字图像，每张图像大小为 28x28 像素。通过使用 MNIST 数据集训练网络、测试网络，测试全连接神经网络性能。

实验步骤：（不要求罗列完整源代码）

1、加载 MNIST 数据集，并将其转换为 PyTorch 张量格式。
2、构建全连接网络模型，包括输入层、两个隐藏层和输出层。
3、定义损失函数（交叉熵损失）和优化器（Adam 优化器）。
4、训练模型，通过迭代更新模型参数以最小化损失函数。
5、在测试集上评估模型性能，计算分类准确率。

1、加载数据集：

使用 torchvision.datasets 中的 MNIST 类加载 MNIST 数据集，并将其转换为 PyTorch 张量格式。同时，使用 DataLoader 类创建数据加载器，以便在训练过程中批量读取数据。

```
import os
os.chdir ('D:/大学/大三上/深度学习/实验课/实验 3')
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy
from torchvision  import datasets as dsets, transforms
import matplotlib.pyplot as plt
train_data = dsets.MNIST(root = 'D:/大学/大三上/深度学习/实验课/实验 3', train = True,
```

```
                        transform = transforms.ToTensor(), download = True)


test_data = dsets.MNIST(root = 'D:/大学/大三上/深度学习/实验课/实验 3', train = False,
                        transform = transforms.ToTensor())
batch_size = 10
train_gen = torch.utils.data.DataLoader(dataset = train_data,
                                        batch_size = batch_size,
                                        shuffle = True)

test_gen = torch.utils.data.DataLoader(dataset = test_data,
                                       batch_size = batch_size,
                                       shuffle = False)
```

2、构建全连接网络模型：

定义一个继承自 torch.nn.Module 的类 FullyConnectedNN，并在其中实现全连接网络的结构。网络包括一个输入层、两个隐藏层和一个输出层，其中隐藏层使用 ReLU 激活函数。

```
class FullyConnectedNN(nn.Module):
    def __init__(self, input_size, hidden_size1, hidden_size2, output_size):
    # def __init__(self, input_size, output_size):
        super(FullyConnectedNN, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size1)  # 输入层到第一个隐藏层
        self.fc2 = nn.Linear(hidden_size1, hidden_size2)  # 第一个隐藏层到第二个隐藏层
        self.fc3 = nn.Linear(hidden_size2, output_size)  # 第二个隐藏层到输出层
        # self.fc = nn.Linear(input_size, output_size)
    def forward(self, x):
        x = F.relu(self.fc1(x))  # 使用 ReLU 激活函数
        x = F.relu(self.fc2(x))
        x = self.fc3(x)  # 输出层不使用激活函数
        # x = F.relu(self.fc(x))
        return x
```

3、定义损失函数和优化器：

使用交叉熵损失函数作为模型的损失函数，并使用 Adam 优化器来优化模型参数。

```
# 设置模型参数
input_size = 28 * 28  # MNIST 图像大小为 28x28
hidden_size1 = 128
hidden_size2 = 64
output_size = 10  # MNIST 有 10 个类别

# 实例化模型
model = FullyConnectedNN(input_size, hidden_size1, hidden_size2, output_size)
# model = FullyConnectedNN(input_size, output_size)
```

```
# print(model)
device = 'cpu'
# if torch.cuda.is_available():
#     model.cuda()
#     device = 'cuda:0'

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

4、训练模型：
通过迭代训练数据，更新模型参数以最小化损失函数。在每个 epoch 结束后，可以打印出当前的损失值以观察训练过程。

```
num_epochs = 5

for epoch in range(num_epochs):
    for i, (images, labels) in enumerate(train_gen):
        images = images.view(-1, 28*28).to(device)  # 将图像和标签移动到设备
        labels = labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

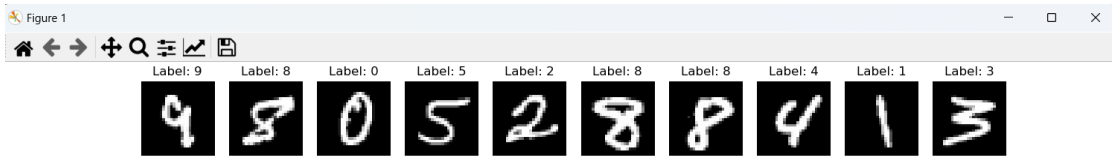
        if (i+1) % 100 == 0:
            print(f'Epoch [{epoch+1}/{num_epochs}], Step [{i+1}/{len(train_gen)}], Loss:
{loss.item():.4f}')
```

5、评估模型：
在测试集上评估模型的性能，计算分类准确率。

```
model.eval()
with torch.no_grad():
    correct = 0
    total = 0
    for images, labels in test_gen:
        images = images.view(-1, 28*28).to(device)
        labels = labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)

    print(f'Accuracy of the model on the 10000 test images: {100 * correct / total:.2f}%')
```

结论分析与体会：
结果如下图




```
Epoch [5/5], Step [4400/6000], Loss: 0.1726
Epoch [5/5], Step [4500/6000], Loss: 0.0554
Epoch [5/5], Step [4600/6000], Loss: 0.0023
Epoch [5/5], Step [4700/6000], Loss: 0.0013
Epoch [5/5], Step [4800/6000], Loss: 0.0029
Epoch [5/5], Step [4900/6000], Loss: 0.3522
Epoch [5/5], Step [5000/6000], Loss: 0.0039
Epoch [5/5], Step [5100/6000], Loss: 0.0365
Epoch [5/5], Step [5200/6000], Loss: 0.0163
Epoch [5/5], Step [5300/6000], Loss: 0.0800
Epoch [5/5], Step [5400/6000], Loss: 0.1816
Epoch [5/5], Step [5500/6000], Loss: 0.0002
Epoch [5/5], Step [5600/6000], Loss: 0.0012
Epoch [5/5], Step [5700/6000], Loss: 0.0087
Epoch [5/5], Step [5800/6000], Loss: 0.3984
Epoch [5/5], Step [5900/6000], Loss: 0.0007
Epoch [5/5], Step [6000/6000], Loss: 0.0034
Accuracy of the model on the 10000 test images: 97.32%
```

通过本次实验，构建了全连接网络模型，并完成了对 MNIST 手写数字数据集的训练和测试。在训练过程中，观察到了损失值随着 epoch 的增加而逐渐减小，这表明模型正在逐渐学习到数据的特征。在测试集上，模型取得了较高的分类准确率，验证了模型的有效性。

在实验过程中，遇到了如模型过拟合、训练速度较慢等问题。通过调整模型结构（如增加正则化项、使用 Dropout 等）和优化器参数（如调整学习率、动量等），改善了模型的性能。

就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1—3 道问答题：
问：如何调整模型的超参数？
答：尝试不同的超参数组合来寻找最优的模型配置。